

M

V

C

SISTEMA DE BIBLIOTECA

Projeto Integrador — Entrega Parcial 2

Estrutura MVC e Sistema de Rotas

Aluno: Hezron Daniel M C Canturil

Polo: Sobradinho — BA | Equipe: Individual

github.com/SonecoO/sistema-biblioteca

PHP 8+

MySQL

MVC

XAMPP

GitHub

Identificação	
Aluno	Hezron Daniel M C Canturil
Polo	Sobradinho — BA
Entrega	Entrega Parcial 2 — Semana 5 (30 pts) — Estrutura MVC e Rotas
Repositório	https://github.com/SonecoO/sistema-biblioteca

1. Objetivo da Entrega

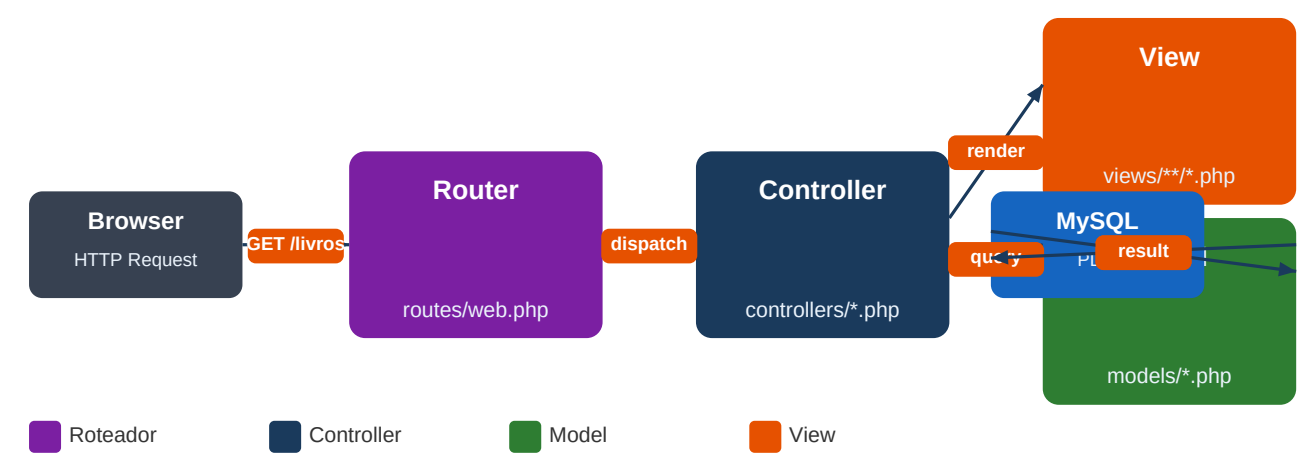
A Entrega Parcial 2 tem como objetivo implementar a **base arquitetural** da aplicação, estabelecendo a estrutura MVC (Model-View-Controller), o sistema de rotas, os controllers iniciais e as views iniciais. Nesta etapa não há ainda conexão ativa com o banco de dados — isso será feito na EP3. O foco é garantir que a aplicação já navegue corretamente entre as páginas e que a arquitetura esteja bem organizada para as próximas entregas.

2. Arquitetura MVC Implementada

O padrão MVC divide a aplicação em três camadas com responsabilidades distintas:

Camada	Responsabilidade	Localização
Model	Representa os dados e regras de negócio. Faz a comunicação com o banco de dados via PDO.	models/
View	Exibe as informações ao usuário. Contém apenas HTML + PHP.	views/
Controller	Recebe a requisição, chama o Model e retorna a View adequada.	controllers/
Router	Analisa a URL e direciona para o Controller/método correto.	routes/web.php

2.1 Diagrama do Fluxo MVC



3. Estrutura de Arquivos do Projeto

A organização abaixo segue as convenções do padrão MVC e facilita a manutenção e evolução do sistema ao longo das próximas entregas.

```
sistema-biblioteca/
├── app/
│   ├── controllers/
│   │   ├── Controller.php
│   │   ├── HomeController.php
│   │   ├── LivroController.php
│   │   ├── AutorController.php
│   │   ├── CategoriaController.php
│   │   └── AuthController.php
│   ├── models/
│   │   ├── Database.php
│   │   ├── Model.php
│   │   └── Livro.php
│   ├── views/
│   │   ├── layout/ (header, footer, app)
│   │   ├── livros/ (index, form, show)
│   │   ├── autores/ (index, form)
│   │   ├── categorias/ (index, form)
│   │   ├── auth/ (login)
│   │   └── errors/ (404)
│   ├── helpers.php
│   └── config/
│       ├── app.php
│       ├── database.php
│       └── autoload.php
├── public/
│   ├── index.php
│   ├── .htaccess
│   └── uploads/capas/
├── routes/
│   └── web.php
└── README.md
```

4. Sistema de Rotas

O roteamento é feito pelo arquivo **routes/web.php**, que intercepta a URL, extrai o controller e o método, valida contra uma tabela de rotas permitidas e instancia o controller correto — sem depender de framework externo.

routes/web.php — Tabela de rotas definidas

```
// Rotas disponíveis no sistema
$routes = [
    'HomeController'      => ['index'],
    'LivroController'     => ['index', 'criar', 'salvar', 'editar', 'atualizar', 'deletar', 'show'],
    'AutorController'     => ['index', 'criar', 'salvar', 'editar', 'atualizar', 'deletar'],
    'CategoriaController' => ['index', 'criar', 'salvar', 'editar', 'atualizar', 'deletar'],
    'AuthController'     => ['login', 'autenticar', 'logout'],
];

// Exemplos de URLs geradas:
// http://localhost/sistema-biblioteca/public/livro/index
// http://localhost/sistema-biblioteca/public/livro/criar
// http://localhost/sistema-biblioteca/public/auth/login
```

4.1 Redirecionamento via .htaccess

O arquivo **public/.htaccess** redireciona todas as requisições para **index.php**, implementando o padrão Front Controller:

public/.htaccess

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^(.*)$ index.php?url=$1 [QSA,L]
```

5. Controllers Implementados

Todos os controllers herdam de **Controller.php** (classe base), que fornece os métodos `view()` e `render()` para carregar as views.

Arquivo	Responsabilidade
Controller.php (base)	Classe abstrata. Métodos <code>view()</code> e <code>render()</code> para carregar views.
HomeController.php	Página inicial do sistema após login.
LivroController.php	index / criar / salvar / editar / atualizar / deletar / show
AutorController.php	index / criar / salvar / editar / atualizar / deletar
CategoriaController.php	index / criar / salvar / editar / atualizar / deletar
AuthController.php	login / autenticar / logout — autenticação real na EP5

app/controllers/Controller.php — classe base

```
abstract class Controller {
    protected function render(string $view, array $data = []): void {
        $data['content_view'] = $view;
        $this->view('layout/app', $data);
    }
    protected function view(string $view, array $data = []): void {
        extract($data);
        require_once APP . '/views/' . $view . '.php';
    }
}
```

6. Models Implementados

A camada de Model usa o padrão **Singleton** para a conexão PDO, garantindo uma única instância de conexão durante toda a requisição.

app/models/Database.php — Singleton PDO

```
class Database {
    private static ?PDO $instance = null;
    public static function getInstance(): PDO {
        if (self::$instance === null) {
            $dsn = 'mysql:host='.DB_HOST.';dbname='.DB_NAME.';charset=utf8mb4';
            self::$instance = new PDO($dsn, DB_USER, DB_PASS, [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            ]);
        }
        return self::$instance;
    }
}
```

7. Views Implementadas

As views seguem uma hierarquia de layout: todas as páginas internas utilizam o **layout/app.php**, que inclui header e footer automaticamente. A tela de login possui layout próprio (sem navbar).

Arquivo	Descrição
layout/header.php	Navbar, CSS global, abertura do HTML
layout/footer.php	Rodapé e fechamento do HTML
layout/app.php	Container principal com flash messages
auth/login.php	Tela de login (layout independente)
home/index.php	Página inicial com botões de acesso rápido
livros/index.php	Listagem do acervo (tabela — dados na EP3)

livros/form.php	Formulário de cadastro/edição com upload de capa
livros/show.php	Detalhes de um livro
autores/index.php	Listagem de autores
autores/form.php	Formulário de autor
categorias/index.php	Listagem de categorias
categorias/form.php	Formulário de categoria
errors/404.php	Página de erro 404

8. Como Executar no XAMPP

1	Baixe o arquivo sistema-biblioteca-EP2.zip e extraia dentro de C:\xampp\htdocs\
2	Inicie o Apache e o MySQL no painel do XAMPP
3	Abra o navegador e acesse: http://localhost/sistema-biblioteca/public
4	O sistema vai redirecionar para a tela de login
5	Clique em Entrar (qualquer dado) — autenticação real será na EP5
6	Navegue pelos menus: Acervo, Autores, Categorias

9. Próximas Entregas

Entrega	Tema	Conteúdo	
EP3 — Sem. 7	CRUD Inicial	Conexão PDO + Create e Read de Livros com MySQL	■
EP4 — Sem. 9	CRUD Completo	Update, Delete, validações e mensagens de feedback	■
EP5 — Sem. 12	Autenticação	Login real, hash de senha, perfis e proteção de rotas	■
Final — Sem. 16	Projeto Final	Deploy, documentação completa e vídeo demonstrativo	■